

Reviewer Pack — Minimal Representation Rank of Matroids Associated with Bool...

Entry 4c89bbb0f747 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 04:03:24 UTC

Minimal Representation Rank of Matroids Associated with Boolean Functions and Tensor Rank of Permutations

Entry ID: 4c89bbb0f747 Verdict: INCONCLUSIVE Recorded: 2026-05-22 04:03:24 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Boolean Function Complexity Theory

Statement:

{‘main’: ‘For any n -ary boolean function f , the minimal representation rank $\rho(f)$ of its associated matroid is lower bounded by the tensor rank $\tau(n)$ of a permutation corresponding to the sign of f .’, ‘corollary1’: ‘Consequently, for all $n \geq 3$, if $\tau(n) \leq c * n^{(1.5 - \epsilon)}$, then there exists a boolean function with representation rank $\rho(f) \leq c * n^{(1.5/2)}$ ’, ‘corollary2’: ‘For any $\epsilon > 0$ and $m < n^{(1.5)}$, if $\tau(m) \leq c * m^{(1.5 - \epsilon)}$, there is no boolean function f with representation rank $\geq c * (n/m)^{(1.5/2)}$.’}

Rationale (proposer’s reasoning):

{‘linkage’: ‘Algebraic combinatorics provides tools to study the structure of matroids, which are associated with boolean functions through their Tutte polynomial. Tensor rank of permutations is a well-studied quantity in algebraic geometry that could capture the complexity of these structures.’, ‘potential_structure’: ‘This conjecture aims to expose a deep connection between the abstract structure of matroids and the geometric properties of permutations, potentially leading to new insights into the complexity of boolean functions.’, ‘separation_implication’: ‘If true, this conjecture would provide a direct link between representation theory and geometric complexity theory, offering a novel approach to separating complexity classes in computational complexity.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 936502347fceed63

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the correlation coefficient between $\rho(f)$ and $\tau(n)$ is greater than or equal to 0.8 for at least 24 out of 30 seeds, and no seed produces a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal representation rank" AND "Boolean function complexity theory"
- "tensor rank of permutations" AND "Algebraic combinatorics" - "matroid associated with Boolean functions" AND "representation rank lower bound"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def boolean_function_to_permutation(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("f must be a list of length 2^n")
        return [f(i).index(1) for i in range(n)]

    def tensor_rank(permutation):
        n = len(permutation)
        rank = n
        for i in range(n):
            if permutation[i] == i:
                continue
            j = permutation[i]
            permutation[i], permutation[j] = permutation[j], permutation[i]
            rank -= 1
        return rank

    def minimal_representation_rank(f):
        n = int(math.log2(len(f)))
        matroid_matrix = [[0]*n for _ in range(n)]
```

```

    for i, x in enumerate(f):
        if x == 1:
            for j in range(n):
                matroid_matrix[i%n][j] = (i//n) ^ (j//n)
    rank = n
    for i in range(n):
        if matroid_matrix[i][i] == 0:
            continue
        for j in range(n):
            if j != i and matroid_matrix[j][i] != 0:
                for k in range(n):
                    matroid_matrix[j][k] ^= matroid_matrix[i][k]
                rank -= 1
    return rank

n = random.randint(5, 40)
f = generate_boolean_function(n)

try:
    permutation = boolean_function_to_permutation(f)
    tensor_rank_value = tensor_rank(permutation)
    representation_rank_value = minimal_representation_rank(f)

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient(tensor_rank_value, representation_rank_value),
        "instances_tested": 1,
        "conjecture_holds": correlation_coefficient(tensor_rank_value, representation_rank_value) == 1,
        "counterexample": ""
    }
except Exception as e:
    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": str(e)
    }

def correlation_coefficient(x, y):
    n = len(x)
    if n != len(y):
        raise ValueError("x and y must have the same length")

    mean_x = sum(x) / n
    mean_y = sum(y) / n

    numerator = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y))
    denominator = math.sqrt(sum((xi - mean_x)**2 for xi in x)) * math.sqrt(sum((yi - mean_y)**2 for yi in y))

    if denominator == 0:
        return None

    return numerator / denominator

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or list(range(3, 157)) # First 30 primes

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

if all(r["conjecture_holds"] for r in results):
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(r["conjecture_holds"] is False and r["counterexample"] != "" for r in results):
    counterexamples = [r["counterexample"] for r in results if not r["conjecture_holds"]]
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{' '.join(counterexamples)}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16977
2	preregistration	ollama_remote	glm4:latest	0	0	9447
3	novelty	ollama_remote	glm4:latest	0	0	8290
4	novelty	ollama_remote	glm4:latest	0	0	8943
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17194

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12841
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9591
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11627
9	judge	ollama_remote	glm4:latest	0	0	14995

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109905 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/4c89bbb0f747.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4c89bbb0f747.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4c89bbb0f747.tar.gz (if generated)